

Competitive Programming

Section 43: String Algorithms I

Competitive Programming Course

2026-07-10

Table of contents

1	Prefix-function thinking	3
1.1	What the prefix function stores	3
1.2	KMP matching	3
1.3	Borders	4
1.4	Minimal period	4
1.5	Why KMP is linear	4
1.6	Matching example	4
1.7	Debugging KMP	5
2	Exercises	6
2.1	Upsolve foundations	6

1 Prefix-function thinking

The prefix function $\text{pi}[i]$ is the length of the longest proper prefix of s that is also a suffix of $s[0..i]$. KMP uses it to avoid rechecking characters after a mismatch. Borders and periods come from repeatedly following pi links.

1.1 What the prefix function stores

For $s = \text{ababa}$, the prefix function is:

i	0	1	2	3	4
$s[i]$	a	b	a	b	a
pi	0	0	1	2	3

At $i = 4$, the longest proper prefix equal to a suffix of ababa is aba , so $\text{pi}[4] = 3$.

The update rule is:

```
j = pi[i-1]
while j > 0 and s[i] != s[j]:
    j = pi[j-1]
if s[i] == s[j]:
    j += 1
pi[i] = j
```

The while loop follows shorter and shorter borders. It never jumps randomly: if a border of length j fails, the next possible border length is the border of that border, namely $\text{pi}[j-1]$.

1.2 KMP matching

To find occurrences of pattern p in text t , build:

```
p + separator + t
```

The separator must be a character not appearing in either string. Whenever $\text{pi}[i] == \text{len}(p)$, an occurrence ends at i in the combined string.

The starting position in the text is:

```
i - 2*len(p) + 1
```

The formula comes from subtracting the pattern, separator, and the pattern length ending at i . In this course the output positions are 1-based.

1.3 Borders

A border is a string that is both a proper prefix and a suffix. If k is a border length of the whole string, then $\text{pi}[k-1]$ is the next shorter border. Therefore all borders are found by:

```
k = pi[n-1]
while k > 0:
    record k
    k = pi[k-1]
```

Reverse the list if the statement asks for increasing order.

1.4 Minimal period

Let n be the string length and $b = \text{pi}[n-1]$. The longest border has length b , so the candidate period is:

```
p = n - b
```

If n is divisible by p , the string is repetitions of the first p characters. Otherwise no shorter period covers the whole string, and the answer is n .

Example:

```
s = ababab
pi[n-1] = 4
p = 6 - 4 = 2
answer = 2
```

1.5 Why KMP is linear

The index i only moves forward. The border pointer j can increase by at most one per i , and every decrease follows a prefix link. Across the whole run there are only $O(n)$ increases and $O(n)$ decreases, so the algorithm is linear.

1.6 Matching example

Pattern:

```
aba
```

Text:

```
ababa
```

Combined string:

```
aba#ababa
```

Whenever the prefix function reaches 3, the pattern has just ended. That happens for the text endings at positions corresponding to starts 1 and 3. The overlapping occurrence is found naturally because after a full match KMP falls back to the longest border of the pattern, not to zero.

This is the main advantage over naive matching: characters that are known to match are not rechecked.

1.7 Debugging KMP

Test these cases before trusting a prefix-function solution:

```
pattern longer than text
pattern equals text
overlapping matches: pattern=aba, text=ababa
all same character: pattern=aaa, text=aaaaa
no matches at all
```

For borders, test strings with a border chain:

```
aaaa -> borders 1,2,3
ababab -> borders 2,4
abc -> no borders
```

If a position formula is wrong, overlapping matches usually expose it.

2 Exercises

2.1 Upsolve foundations

2.1.1 Prefix function vocabulary

For a string s , $\text{pi}[i]$ is the length of the longest **proper** prefix of $s[0..i]$ that is also a suffix of that same prefix. Proper means it cannot be the entire substring. A length, rather than a copied substring, is enough: the candidate border ending at $i-1$ has length $\text{pi}[i-1]$.

To extend it with $s[i]$, compare $s[i]$ with $s[j]$, where $j=\text{pi}[i-1]$. On a mismatch, the next possible border is the border of that border, of length $\text{pi}[j-1]$. This is the failure link. Do not restart comparison at zero unless all failure links have been exhausted.

```
pi(s):
    pi[0] = 0
    for i = 1..n-1:
        j = pi[i-1]
        while j > 0 and s[i] != s[j]:
            j = pi[j-1]
        if s[i] == s[j]:
            j += 1
        pi[i] = j
```

2.1.2 Trace: prefix function of ababa

i	character	candidate j before/after fallback	pi[i]
0	a	-	0
1	b	0	0
2	a	0 -> 1	1
3	b	1 -> 2	2
4	a	2 -> 3	3

The final $\text{pi}[4]=3$ says **aba** is the longest proper border. Following its link gives $\text{pi}[2]=1$, so **a** is the other proper border.

2.1.3 KMP is an indexing problem

For pattern p of length m and text t , build $p + \text{separator} + t$, where the separator is outside the input alphabet. When $\text{pi}[i] == m$, a match ends at combined index i . The first text character is at combined index $m+1$, so the 1-based match start is $i - 2*m + 1$.

For $p=\text{aba}$, $t=\text{abacaba}$, matches end at combined indices 6 and 10, producing starts 1 and 5. This formula also retains overlapping matches: **ana** in **anana** occurs at positions 1 and 3.

2.1.4 Borders and periods are the same link structure

Every proper border length of s is found by:

```
k = pi[n-1]
while k > 0:
    output k
    k = pi[k-1]
```

The first candidate period is $p=n-\text{pi}[n-1]$. It is a true tiling period only when $n \% p == 0$. For **ababab**, $p=2$ divides 6. For **ababa**, the same candidate is 2 but does not divide 5, so the minimal repeating block length is 5, not 2.

2.1.5 Proof and implementation habits

The loop invariant is: before processing position i , j is the longest border of $s[0..i-1]$ that has not been disproved by the new character. Every fallback preserves the possibility of matching a shorter border; every success extends it by one. Since j only rises after a successful character comparison and every fallback lowers it, total fallback work is linear.

In C++, use `vector<int>` for `pi`; in Python use a list of integers. Never make the separator a character that can occur in input. If the alphabet is not controlled, run KMP directly on pattern and text rather than concatenating a string.

2.1.6 Tests and formative checks

1. Compute `pi` for **aaaaa**. Answer: 0 1 2 3 4; this checks chained borders.
2. List all proper borders of **abcabca**. Answer: 1 and 4.
3. Explain why **ababa** has candidate period 2 but answer 5. The divisibility check is the missing condition.
4. Test a pattern longer than text, a pattern equal to text, and overlapping matches before trusting an occurrence implementation.

Run the local reference suite with:

```
CP_TARGET=solution python3 sections/43_string_algorithms_i/check.py
```

Before submission, state which of these three interpretations you use: a full prefix-function value is a matched pattern, a prefix link is a shorter border, or the final border determines a candidate period.

- A. KMP Occurrences: Print the number and 1-based positions of all occurrences of pattern in text.
- B. All Borders: Print all proper border lengths of a string in increasing order.
- C. Minimal Period: Print the shortest period length if the string is repetitions of a block, otherwise n .